

Fall 2026

Lab 1: Introduction to Reachy Mini

Due: September 10, 2026 at 11:00PM

Assignment Type: Group (Up to 3 members)

Submit code and report via GitHub URL

Total Points: 18 points

0. Overview and Learning Goals

Programming robots is hard despite recent hype around humanoid robots that can clean your home or cook food. Nevertheless, software tools and hardware have advanced in recent years, enabling human-robot interaction (HRI) developers and researchers to focus on problems more pressing than debugging code, and noisy sensor data leading to inaccurate perception outputs. To understand the existing limitations in robotic technology, this lab will expose you to introductory materials to familiarize students with the Reachy Mini platform and challenge you to explore failures in existing applications. In this lab, you will use Python and the Reachy Mini SDK to control simulated and physical robot behavior. Reachy Mini uses a daemon to communicate with its motors, sensors, applications, and simulation environment.

The **learning outcomes** of Lab 1 are:

- Installation Reachy and dependencies
- Setup development environment
- Run an app in the MuJoCo simulation
- Create your first ReachyMini application
- Investigation and reflect on Reachy Mini failures modes

See the ‘**Reachy Mini Helper Guide**’ on Canvas for FAQs.

1. Create a Team GitHub Repository

Do not create a public repository. Add all team members and verify access before beginning shared work.

1.1 Configure GitHub authentication

[Generate a new SSH key](#) and add it to the ssh-agent. The SSH key enables you to authenticate your GitHub account locally in order to pull, push, and merge code to your GitHub repository (**Remember to select the Linux OS in GitHub Documents guide**).

1.2 Clone and organize the repository

- We recommend creating a team to share code with your team.
- At “Manage Access”, click “Create team”. Add a team name and add members. Then add this team to your repository at the “Manage Access” page.
- Clone the team repository on the VM by opening a terminal window and using the following command: `git clone <SSH URL>`

- Commit changes to the repository using this command: ``git commit -m <add a message to describe code changes>``
- Push code changes to the repository by running this command in the terminal ``git push <add command>``

Deliverables (3 points):

- Team repository with complete structure, well-documented README, .gitignore, reproducible instructions, and team commit history.
- The final repository contains no credentials, placeholder submission text, unrelated files, or broken run instructions.

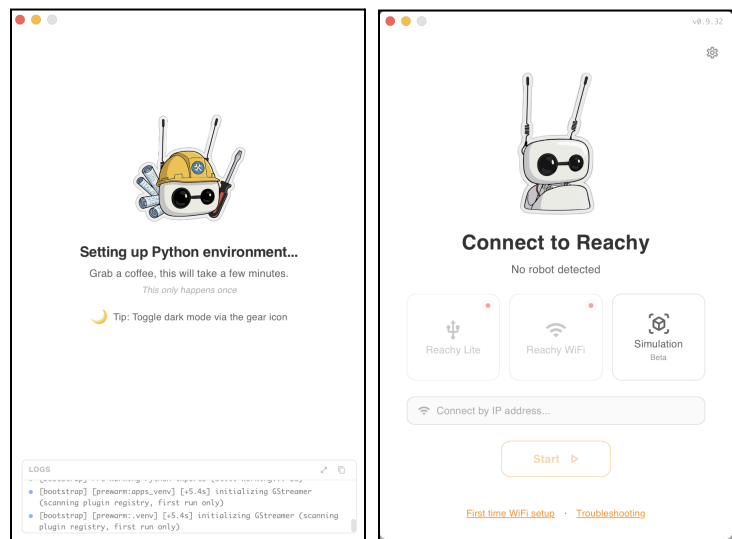
2. Install Reachy Mini Control

2.1. Download Reachy Mini desktop application.

Download the current Reachy Mini Control desktop application from the official Reachy Mini website. Choose the installer for your operating system and follow the on-screen instructions.

2.2. Open the ‘Reachy Mini Control’ app.

Open Reachy Mini Control. Allow local-network access so the application can discover the simulator and physical robot. Grant microphone or camera access only when the application you are testing requires it.



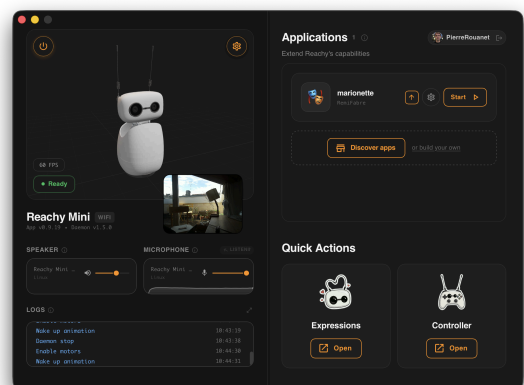
Note: Mac devices require access to be granted; otherwise, the app won't launch its homepage.

2.3. Click ‘Simulation Beta’ and ‘Start’

Select Simulation Beta and click Start. Keep Reachy Mini Control open while the simulator initializes. A MuJoCo window should appear with a 3D model of Reachy Mini. If the desktop simulation does not start, use the SDK-based simulation path in Section 4.2 and consult the official simulation guide.

2.4. The following window will pop up. Sign into or create a Hugging Face account.

When prompted, sign in to an existing Hugging Face account or create one.

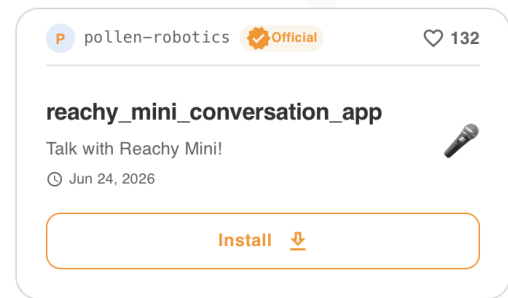


2.5 Run an app! Click Discover Apps. For example, run the reachy_mini_conversation_app.

Open Discover Apps and select a compatible application. The `reachy_mini_conversation_app` is one example. Record the application name and version when available.

Note: If the app repeatedly crashes, update the dameon version in simulation settings.

- Predict the application's inputs, outputs, and intended interaction before launching it.
- Run one complete interaction cycle in simulation.
- Stop the application from the dashboard and confirm that it has fully stopped.



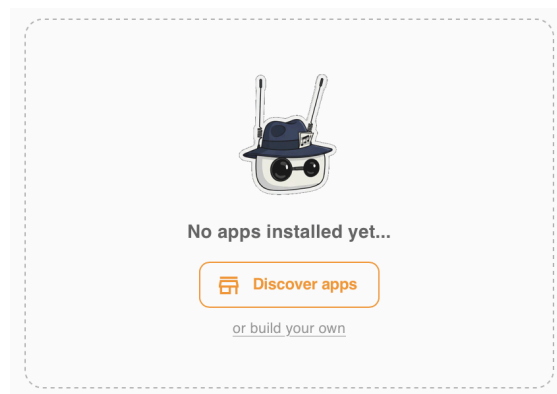
Deliverables (2 points):

- Screenshot showing Reachy Mini Control connected to Simulation Beta with the MuJoCo window visible.
- Screenshot showing the selected application running, with its name and author recorded in the report.
- A short setup record listing your computer operating system and architecture, Reachy Mini Control version, and any installation issue/resolution.

3. Run an app in Simulation mode

3.1 Browse, select, and run an *app* in Simulation mode

Browse community applications either through Discover Apps in Reachy Mini Control or through the [Reachy Mini community applications page](#). Select one application that can be run in simulation for systematic observation. You may reuse the application from Section 2.5. Before running it, write a one-sentence prediction of the user action, robot input, application decision, and robot output.



3.2 Observe one complete interaction

- Start the application and complete at least one intended interaction cycle.
- Record the user action or response during the interaction and compare it with the response the application appears to expect. Note any point where the user's behaviour or interpretation differs from the expected interaction.
- Record what the simulator represents well: movement sequence, timing, state changes, or interface feedback.
- Record what the simulator cannot establish: motor behavior under load, audio direction, camera quality, network latency, physical clearance, or a person's interpretation of the motion.

- Stop the application before proceeding to teleoperation.

3.3 Teleoperation in Simulation mode

Use Reachy Mini Control or the simulator dashboard to operate the head, body yaw, and antennas. Make small movements, observe the resulting pose, and return the robot to neutral. Compare deliberate smooth movement with faster direct control, but use conservative speeds and amplitudes.

Deliverables (3 points):

- A short clip showing the selected community app completing one interaction in simulation.
- A screenshot showing teleoperation of at least two expressive channels and the robot returned to neutral.
- A detailed observation describing the robot's intended interaction, the user's actual response, the robot's resulting behaviour, and two limits for evaluating HRI.

4. Create and Run an App in Simulation

4.1 Verify the Simulation Environment

Start Simulation Beta in Reachy Mini Control or launch the SDK simulator. Confirm that the MuJoCo window and local dashboard are available before creating your app. Keep the simulator/daemon running in its own terminal during development.

4.2 Create a Workspace

Complete the official [Reachy Mini SDK installation guide](#) for your operating system (Mac and Linux OS only; Windows is not supported).

Requirements for this lab:

- Use Python 3.10–3.12; Python 3.12 is recommended.
- Create and activate a virtual environment named `reachy_mini_env`.
- Install Reachy Mini with the MuJoCo simulation dependencies.
- Create an `apps/` directory inside your private course repository.
- Add `reachy_mini_env/` to `.gitignore`.
- Record the following in `README.md`:
 - Operating system and architecture
 - Python version
 - Reachy Mini version
 - MuJoCo version

Before continuing, confirm that the MuJoCo simulation starts successfully by following the [official simulation setup guide](#).

4.3 Create a Reachy Mini application

Follow the “Creating an App” and “Testing Your App” sections of the [official Reachy Mini app-development guide](#).

Requirements:

- Name the application `team_greeting_app`.
- Do not use the `--publish` option.
- The app assistant must be run from a location outside the existing course Git repository.
- After generation, move the complete `team_greeting_app/` folder into:
 - `apps/team_greeting_app/`
- Run the app assistant's validation check.
- Install the application in editable mode.
- Keep the application only in the team's private GitHub repository.

The final application must:

- Extend `ReachyMiniApp` and implement `run(reachy_mini, stop_event)`.
- Coordinate at least two expressive channels: head, body yaw, or antennas.
- Contain three stages: orient toward an implied user, perform a greeting, and return to neutral.
- Check `stop_event` during loops/waits and stop gracefully.
- Print timestamped markers for each stage and expose at least two named motion/timing parameters with documented units.
- Avoid camera, microphone, cloud AI, and collection of personal data for this first app.

4.4 Test the application in MuJoCo

Use the operating-system instructions in the [simulation guide](#) to start the simulator.

Use the official [app-testing instructions](#) to install and run `team_greeting_app`.

Use two terminals:

1. Activate `reachy_mini_env` in both terminals.
2. Keep the simulator or daemon running in the first terminal.
3. Run and stop `team_greeting_app` from the second terminal.
4. Press Control-C to request a normal stop.

Confirm that:

- All three behavioral stages are visible.
- The application exits cleanly.
- No control loop remains running.
- The simulated robot returns to neutral.
- The timestamped log markers match the observed stages.

4.5 Iterate and validate in simulation

- Run at least three complete cycles in MuJoCo and verify that the behavior is repeatable.
- Test a normal stop request during each stage and confirm that the app exits without leaving a control loop running.
- Vary at least two named parameters and record the observed effect on timing and legibility.
- Choose final candidate values and explain why they should transfer safely to the physical robot.
- Return the simulated robot to neutral at the end of every trial.

Deliverables (5 points):

- Complete app source, pyproject.toml, and run instructions under apps/team_greeting_app/.
- Screenshot and a short clip showing all three stages and matching log markers in MuJoCo.
- A simulation test record covering three repeated cycles, stop behavior, and at least two parameter variations.
- A concise rationale for the final candidate parameters, including anticipated physical-robot risks.

5. Define test cases, observe robot behavior, and reflect on failures

Conduct this investigation in the simulator. Return to the community application used in Section 3. A failure is not only a crash. In HRI, failure includes a missed signal, incorrect inference, confusing feedback, unsafe or illegible motion, unexpected delay, or behavior that violates a user's expectation.

Write one research question and change only one independent variable (IV) across at least three simulated test cases: a baseline, a moderate challenge, and a boundary case. For each test, record an observable robot behavior as a measurable dependent variable (DV), such as task completion, response time, correct motion, or feedback. Complete the table below.

- **Baseline:** The normal or default condition which the application is supposed to work
- **Moderate challenge:** A more demanding situation within normal limits and the same IV
- **Boundary case:** The most demanding case for the IV

Possible independent variables include response delay (e.g., 0, 3, and 10 seconds), stop-request timing, motion amplitude, or input variation. Choose one independent variable and hold all remaining conditions constant. Treat any limitation that requires real sensors or human interpretation as a documented limitation rather than moving to the physical robot early.

Test	Condition (IV)	Expected	Observed (DV)	Evidence	Result
1	Baseline			timestamp	pass/fail
2	Moderate challenge			timestamp	pass/fail
3	Boundary case			timestamp	pass/fail

5.1 Failure reflection

Identify First Divergence Point: This is a critical step. The first time the robot's behavior deviates significantly from what you expected, that's your starting point for investigation.

Classify source of failure: Analyze why the behavior diverged. Is it a problem with:

- Sensing/Perception: Does the robot misinterpret the input?
- Decision/Model Logic: Is the robot making the wrong choice?
- Motion/Control: Is the robot's movement incorrect?
- Timing/Network: Is there a delay or communication issue?

- Interface/Feedback: Is the robot misinterpreting the feedback?
- Environment: (simulated) – is there something in the simulation affecting the outcome?
- User Expectation: Is the robot failing to meet the expected behavior of a human?

Evidence vs Inference: Separate evidence from inference: state what you observed, what you think caused it, and your confidence. More specifically, separate observation from your interpretation. “I observed that the robot stopped abruptly.” vs. “I think the robot stopped abruptly because the sensor readings were erratic.” Be careful not to jump to conclusions.

Migration: Propose one technical mitigation (e.g., “Implement a sensor filtering algorithm”) and one interaction-design/feedback mitigation (e.g., “Provide clearer visual feedback to the user when the robot is struggling”).

Limitations: Identify a limitation of your simulation (e.g., “The simulation doesn't account for friction or dynamic effects”) and propose a follow-up test to reduce uncertainty (e.g., “Run the simulation with a more realistic friction model”).

Deliverables (5 points):

- Completed three-case test table with controlled conditions, expected/observed behavior, pass/fail result, and timestamped evidence.
- Screenshots or clips for all three cases, labeled so the grader can match them to the table.
- A brief reflection that distinguishes observation from inference and proposes technical and interaction-design mitigations.
- One limitation and one concrete follow-up test.